

Proposed Enhancement - Gemini CLI

Group Name: The Rat Pack

Aatman Gupta 22ag32@queensu.ca

Caleb Gao 21cg49@queensu.ca

Cory Boys (presenter) 21alb25@queensu.ca

Finnegan MacAdams (presenter) 23bmd@queensu.ca

Jude Campanelli-Henderson (leader) 22nbx2@queensu.ca

Rémy Brown-Vandervecht 22WLHR@queensu.ca

CISC 322

Professor: Bram Adams

TA: Ayesha Babar

Abstract

The purpose of this report is to propose a new enhancement to the Gemini CLI AI agent. We have chosen to add audio features within Gemini CLI to aid users who may have issues typing or reading words. The report also includes two use cases and their sequence diagrams. The first focuses on the use of a developer interacting with the AI to work on code in a conversational manner. The second focuses on the case where a user asks the AI to create a file containing a program that meets requirements taken from a URL. The report then explains the effects of the enhancement on the system. The report also includes two alternatives for realizing our proposed enhancement. We explore implementing the audio feature as a subsystem to the core package, as well as the concept of limiting the features integration through either creating its own package or outsourcing. We had then performed a SAAM analysis on the proposed alternatives to determine which architecture best balances the implementation of voice features. We have included box-and-arrow diagrams to show how our audio enhancements would fit on both high and low-level conceptual architecture. We explain the plan for testing using timer and computer resource monitors. Finally, we have discussed the potential security, maintainability, performance, and usability risks due to our enhancements and their possible solutions.

Introduction

We have proposed an audio interaction feature to enhance the use of Gemini CLI. This includes two new main features: Voice-to-Text and Text-to-Voice. The first allows users to simply speak their input directly into the computer rather than type it. The computer then passes it to a Voice-Speech interface, which converts the voice into input that the CLI can process and send to the Gemini API (still going through the regular processing channels after the Voice-Speech interface). The latter enables the user to read the CLI's output using the same interface. We chose this enhancement to improve the accessibility of Gemini CLIs. Speech-to-text tools can be extremely helpful for people with certain disabilities, impairments, or other conditions.¹ When combined with the Gemini CLI, this would allow many more people to easily access the Gemini API for questions from their terminal. Similarly, text-to-speech is also a helpful tool for the visually impaired.² A tool like Gemini CLI can already remove the barriers to information for people who struggle with reading and finding resources on their own, and this accessibility feature further supports this access to information. A feature like text-to-speech can also improve a user's experience, opening the Gemini CLI to many new and returning users.³ Of all the enhancements we were considering, we found that this would have the greatest positive impact on AI use.

¹ (ysg_adm22)

² (Staff Reporter)

³ (Staff Reporter)

Use Cases Legend

Arrow Types:

————→ Synchronous method call (caller waits for reply)

————→ Asynchronous method call (caller does not wait for reply)

-- --> Return message/callback

Guard [condition]: Decision point

Lifeline Naming:

CliCommandHandler: CLI entrypoint (handles input/display)

AuthManager: Token validation and security

ConfigService: Loads GEMINI.md, settings, context

OrchestrationEngine: Workflow coordinator (your “brain”)

SessionHistory: Conversation state/chat history

ToolRegistry: Executes tools like “run_tests”, “fix_code.”

GeminiApiClient: Gemini model API calls

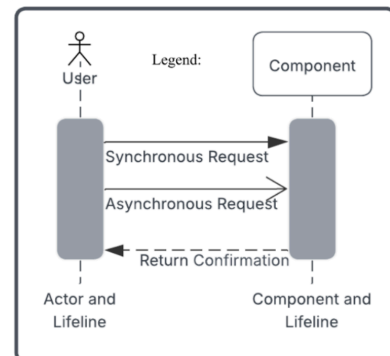


Figure 5. Sequence diagram legend

Use Case 1

The diagram outlines how a developer collaborates with an AI to test and debug their code, making the process feel conversational through voice interaction. It walks through key steps like running tests, analyzing errors, applying fixes, and repeating the process until everything works perfectly.

The interaction kicks off when the developer makes a request, either by typing or speaking through a voice interface. This spoken input gets converted to text and is sent to the CliCommandHandler. From there, it reaches the OrchestrationEngine, which first checks with the AuthManager to verify the user’s access token. Once they’re authenticated, the ConfigService loads the necessary session settings and contextual information. With everything in place, the OrchestrationEngine logs the new prompt in SessionHistory and crafts a structured request that includes session details and context from the ongoing conversation. This request is then passed on to the Gemini model via the GeminiApiClient.

When Gemini indicates that tests need to be run, the OrchestrationEngine seeks confirmation from the developer by sending a message back through the CliCommandHandler. If voice interaction is enabled, the system even reads this message aloud. If the developer gives the thumbs up, the OrchestrationEngine calls the appropriate tool to run the tests. The results

come streaming back, which the OrchestrationEngine adds to the SessionHistory and sends to the CliCommandHandler for the developer to see, with an option for audio playback.

If any tests fail, the process enters a loop to address those issues. In each cycle, the OrchestrationEngine pulls the latest information from SessionHistory, sends it to Gemini for further insights and potential fixes, and activates the necessary tools again through the ToolRegistry. Each new set of results is logged in SessionHistory and presented to the developer via the CliCommandHandler, with audio feedback if it's activated. This looping continues until every test successfully passes.

Once all tests are cleared, the loop ends. Gemini then generates a final summary of the successful run, and relays it back through the CliCommandHandler for the developer's review, with text-to-speech available if desired. If at any point the developer chooses to halt a tool action, the OrchestrationEngine immediately ceases further actions and provides a straightforward status message explaining the cancellation.

In Use Case 2, which involves creating files from web specifications, we decided to leave out the SessionHistory lifeline. This is because it operates on a straightforward single-pass pattern: the history is loaded just once at the start with the `getConversationHistory(sessionId)` function and then appended only once after the tool finishes its task using `appendToolOutput()`. Adding this lifeline would simply duplicate the interactions already shown without providing any new insight into the architecture. By keeping it out, we can maintain the focus on the main workflow of fetching data from the web and writing it to a file.

The diagram below illustrates a use case in which a user asks the AI agent to create a file containing a program that meets the requirements or specifications listed on a website or

online blog, with the URL provided, with the change that the user opts to make requests and interact via audio (speech) input and has interface displays read aloud.

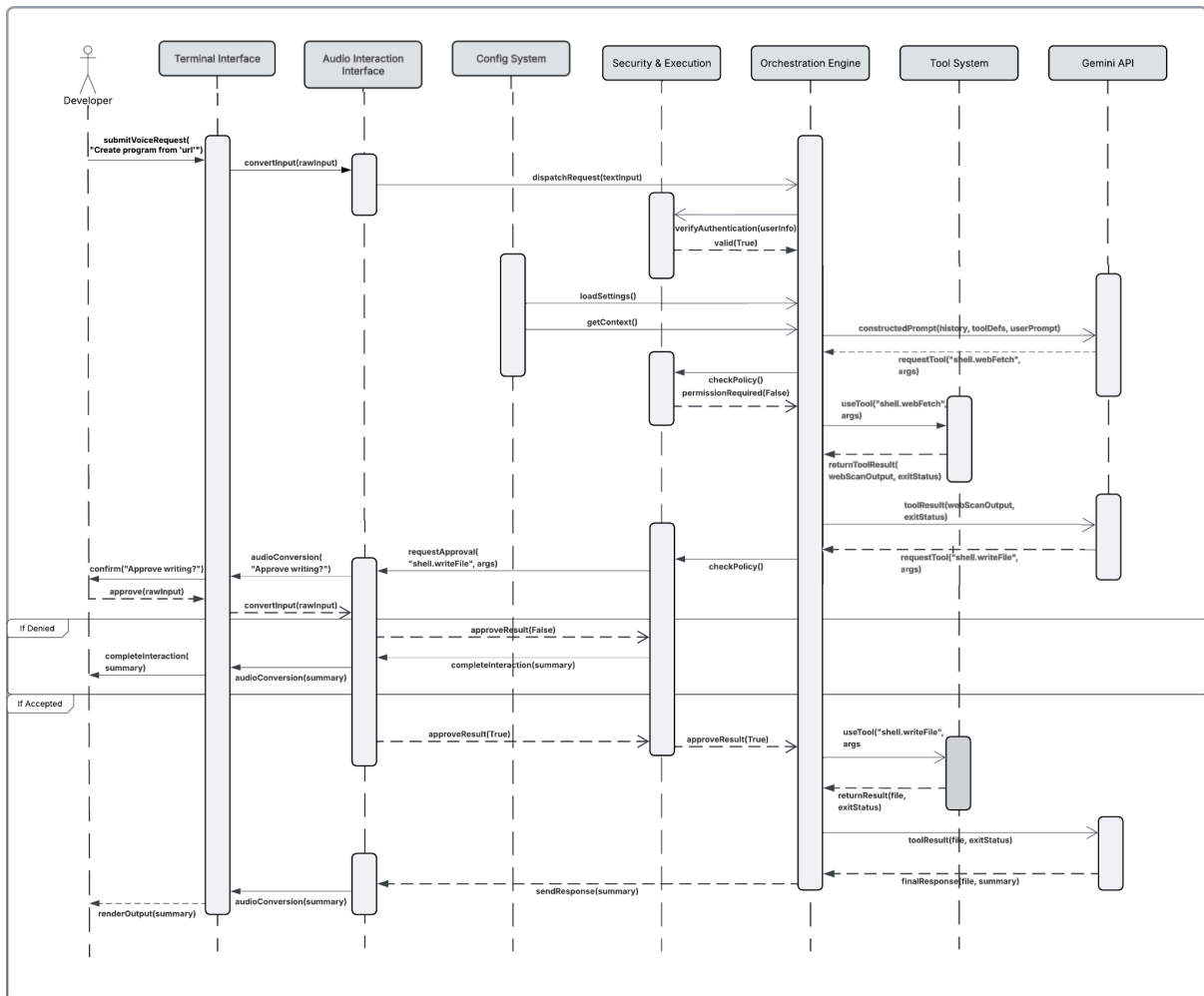
To begin the process, the developer (user) enters the request and URL in the terminal. Unlike in previous use cases, this time it is entered as a voice command, which is sent to the Audio Interaction Interface to convert it into raw text. The Audio Interaction Interface then forwards them to the Orchestrations Engine in the Core package, verifying the user's authentication. The engine then formats the request into a structured prompt for the Gemini AI, adding previous context, available tools, and message history from Configuration settings and session context.

This expanded prompt, which provides the Gemini AI with a clearer understanding of the current development state for use, is then forwarded to the Gemini API, and a response is received.

Gemini's response requires the Web Fetching Tool, which is invoked by the tools component and fetches the requirements from the specified URL. In this use case, we assume, for simplicity, that the request succeeds without interruption and that the URL contains the requested information; if the URL were malformed, the tool would return an error state, prompting the user for additional instructions or an updated URL. Similarly, if the Gemini API received insufficient instructions from the scanned data, it would return a request for more precise specifications. The tool returns the scanned data to the Orchestration engine in the Core package, which then forwards it to the Gemini API again.

Using the scanned web information and the context provided by the Core package, the Gemini API proposes a solution that satisfies all requirements and requests the use of the File System tools to create the file. Since this involves modifying files, the Security Component requires user (developer) approval, which it requests from the terminal. The Core package reroutes this request to the CLI package, which forwards it to the Audio Interaction Interface to convert to an audio file, both of which are sent to the user to inform them about what the AI intends to create or modify and prompts them to approve or deny. If the developer denies, the terminal reports that state to the Core package, which then terminates the process without creating a file and returns a summary message to the user.

If the user approves the proposed file, the Orchestration Engine invokes the File System tools to create the file and write the API's proposed solution to it. When that process is complete, the tools component returns the results to the engine, from which the Gemini API generates its final summary, including the details surrounding the completed file, its capabilities, how it fulfills the requirements outlined for it, and other relevant information and advice regarding the use of the newly created program. The engine then sends this summary to the Audio Interaction Interface, converting it to audio and forwarding to the terminal, which plays it to the user.



Effects of Enhancement

Maintainability:

The implementation of the audio system maintains the modularity of the current Gemini CLI architecture. The packages used for the audio system operate independently from other packages in the system, other than the input received from either the Core or the CLI, there is no necessary integration with lower level components within these packages. This facilitates changes to any layer of the system without affecting the use of the audio system.

Evolvability:

The increased complexity of the system that follows from the implementation of an audio system can complicate evolvability. If the system is hardcoded to specific audio formats then a change in the audio API can require developers to change a lot of the rest of the system. However, since the modularity of the system is well defined and will be preserved when implementing the audio system, there is greater potential for adaptation to different environments and new features.

Testability:

The testing of this feature would complicate testability as there is a new dependency on a remote server, and data validation becomes a concern, especially as it relates to concurrency. The testing for this system would have to include integration tests to ensure that there are no bugs in the boundaries between the CLI, audio component, and the Core. It would also have to set standards for audio ‘correctness’ and incorporate checks that assert that the audio received makes sense in context with the project (i.e. does the mentioned file exist). The addition of the feature decreases testability, but incorporating tests that consider the modularity of the system and are able to isolate audio difficulties from logic issues can improve overall testability.

Performance:

For users who do not choose to use the audio feature, the performance of Gemini CLI stays exactly the same. The data is able to bypass the audio systems and the CLI and Core packages can communicate directly with each other. Otherwise, there would be a small delay as the audio system processes the input or output to send to the next package, but once it has been processed, the system is able to use it as it would otherwise. Although the audio feature is implemented on a remote server, if the user is on a lower end device, the use of the audio feature could cause some device latency as there could be larger audio data transfers.

Alternatives**Alternative A:** Local Processing

The audio feature would be implemented on the client side of the application, in the CLI package. This implementation would involve the audio package only interacting with the CLI component, modifying input and output to either text or speech as needed. This method would improve flexibility and maintainability, as it would allow developers to make changes to the main components of Gemini CLI without having to adjust the audio system or its interactions. However, the heavy communication between the CLI package and the audio package could increase latency and put a high load on the frontend of the system. It would also rely much more on the user’s device, which would require compatibility with many different OSs and could result in increased latency depending on the performance of the device.

Alternative B: Cloud API processing

The audio feature would be an external system communicating with the Core package, mainly communicating with the task delegation system. For speech input from the user, the CLI would send the audio to the Core package, and the task delegation system would initiate the audio system, which would process the data and send a text version back to the task delegation system to process as needed. Similarly, when the user requests an audio version of the produced response or of anything on their screen, either the CLI or Gemini would send the text to the audio system which would produce an audio output. This implementation helps maintain the flexibility of the system as it only really interacts with the Core task management component to receive the data that needs to be transformed. However, there

ends up being some unnecessary back and forth communication between the CLI and Core packages. If the user requests the audio of a piece of text displayed on the screen that wasn't necessarily the most recent output generated by the Core package, then the prompt would have to go through the CLI to the Core and then delegated to the audio system by the Core package.

SAAM Analysis

The Software Architecture Analysis Method provides a structured way to evaluate potential architectural designs against the real needs of stakeholders. This analysis evaluates the two architectural styles for integrating Voice-to-Text and Text-to-voice capabilities for Gemini CLI.

Stakeholders and Non-Functional Requirements

- **End Users** (Developers, Visually Impaired Users): This group would interact with the voice feature daily, so their priorities are about how well it works in practice. *Usability* is critical, because developers dictate commands that are filled with programming syntax, variable names, flags, as well as file paths. A model that mishears 'git rebase' as 'get rebase' produces unusable output. Beyond accuracy, *Performance* is also vital. There should be minimal delay between speaking a command and seeing the CLI respond. And thirdly, *Privacy* is non-negotiable for professional users. Developers may discuss proprietary material while working, and audio containing that content must never be exposed without their explicit consent.
- **System Maintainers** (Developers of Gemini CLI): Maintainers care less about daily usability and more about what happens when the system needs to change. *Maintainability* is a concern because audio processing has a fragmented ecosystem, with microphone access and audio driver APIs behave differently across different operating environments, creating a burden of long-term support.

Evaluating Alternatives against NFRs

- **Privacy:** Alternative A is the clear winner here. Since the processing all happens locally, audio data never travels across a network, and there is no interception risk and no third-party data retention concern. This makes it well suited for developers working on sensitive or proprietary projects. Alternative B proposes real privacy risk. Audio containing potentially sensitive information must be transmitted over the network to the remote server. While HTTPS mitigates interception, the server still receives and processes the raw audio. Opt-in consent flows and clear disclosure policies can reduce the risk, but never fully eliminate it.
- **Performance:** The trade off here runs in opposite directions. Alternative A keeps all processing local, which can strain lower-end devices. Running inference on a speech

model can be computationally demanding, and can affect a user's machine, especially if it is under load from other softwares. Alternative B offloads that computation to the cloud, keeping local resource usage low. The downside is network latency, as the round trip to a cloud API adds a delay. On a slow or unreliable connection, the processing time would be noticeably slower.

- **Usability** (Accuracy): Alternative B has a significant edge here. Cloud providers train their STT models on enormous and continuously updated datasets, which gives them far better handling of programming syntax. Alternative A's lightweight models are optimized for conversational speech, and not for developer commands. They frequently misinterpret coding syntax, which can produce unreliable output and frustrate the user.
- **Maintainability**: Alternative B is considerably easier to maintain. The CLI only needs an API client and standard network libraries. It would not have OS-specific audio processing dependencies. When the remote provider updates their model, the CLI gets the improvements automatically, with no code changes required. Alternative A is the opposite. It requires managing ML model binaries, native inference runtimes, and could even encounter audio driver compatibility issues.

Conclusion

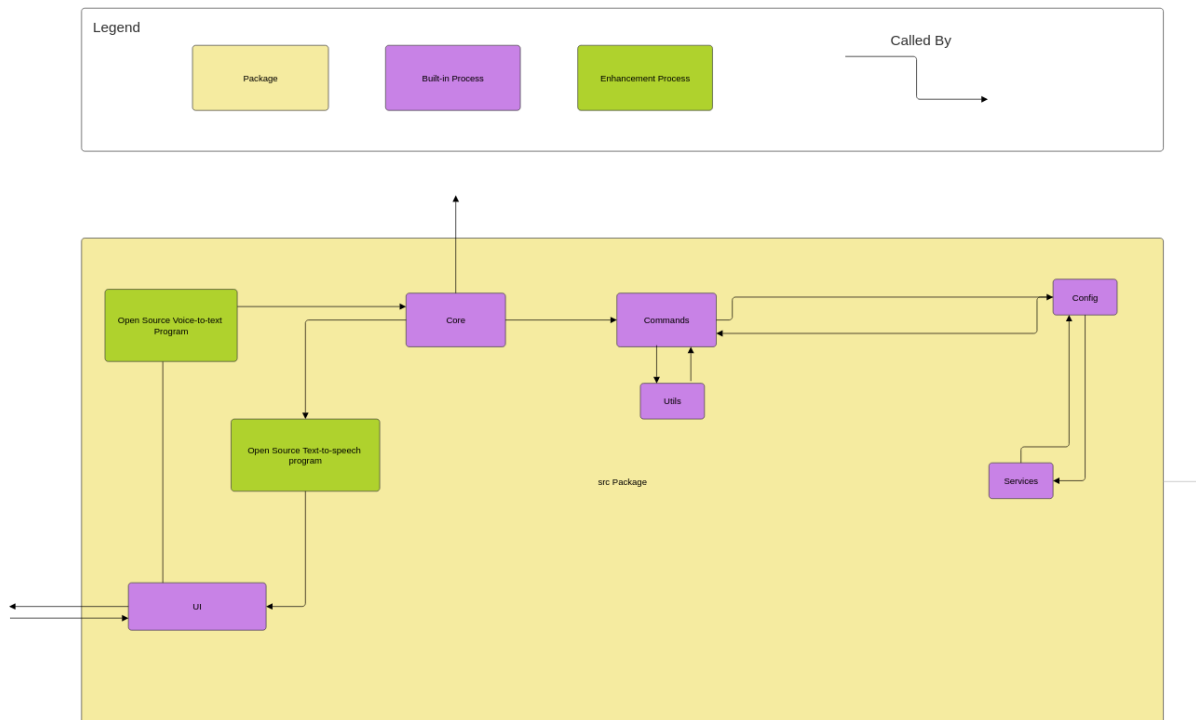
Based on the analysis, Alternative B, *Cloud API Processing* is the stronger architectural choice for this feature. It delivers meaningfully better transcription accuracy for technical vocabulary, imposes a far lower maintenance burden on the dev team, and makes future updates to the underlying model very easy. The primary concern *Privacy*, is a big one, but is addressable through design decisions, such as requiring explicit opt-in, displaying a clear disclosure the first time the feature is activated, and enforcing HTTPS for all API communication. The performance dependency on network connectivity should be acceptable for a large majority of Gemini CLI's users, who work in internet-connected environments. Alternative A remains a viable option for users who specifically require it, and could be offered as a secondary mode if the demand arises.

Effects on the High and Low-level conceptual architectures

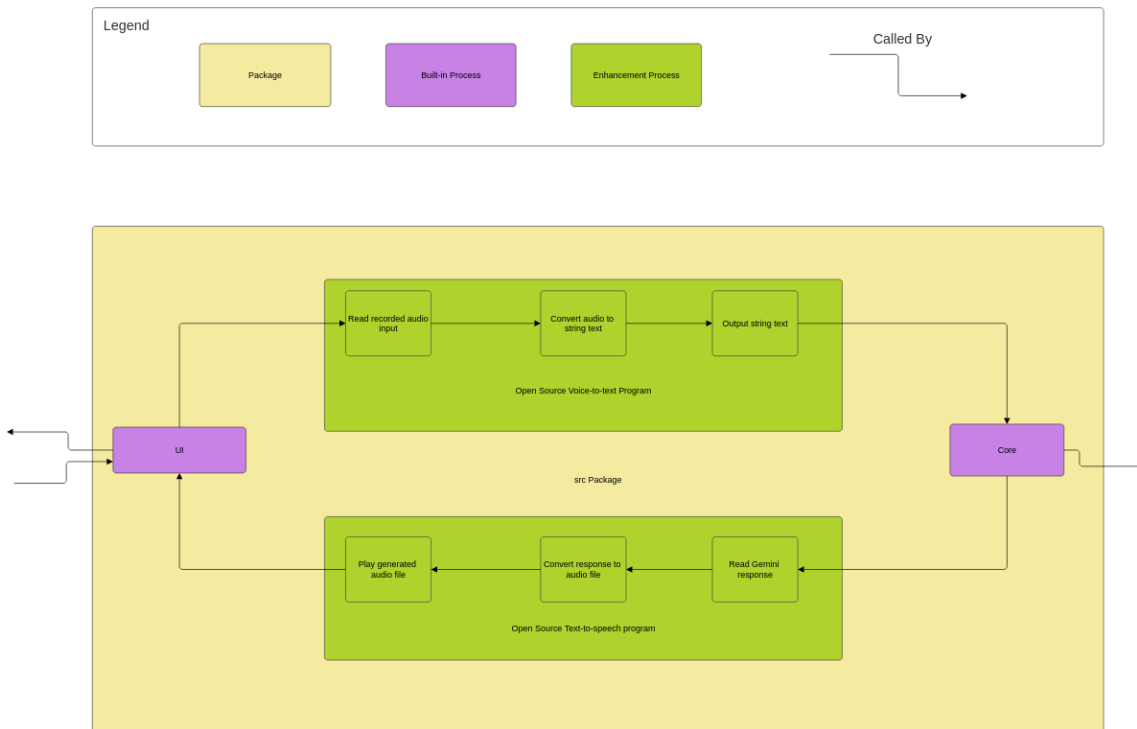
Our enhancement affects Gemini's architecture in a few very simplistic ways. It involves only adding a couple packages along the command pipeline to interpret the user's voice command, and parse the output response to a text-to-speech program.

As depicted by the low-level diagram below, the inbound execution flow to the core package is routed through the UI process, where any typed text would normally be processed and sent to the core. We have added a new process along this route that deploys an open source Voice-To-Text program for converting the user's speech into a parseable text string for the core, as if it were a standard typed input.

On the way back from the Core, where the response is normally delivered straight to the UI, we have also implemented an open source Text-to-Speech program that parses the outgoing response from Gemini to audible speech.



In the High-level diagram below, we can see that when the Speech-to-Text program receives a response from the UI, it reads the recorded audio file produced by the user's microphone, uses a complex algorithm to convert the audio to a text string, and then forwards this output to the Core package as if it were a standard typed input. When returning, the Text-to-Speech program does the reverse; it reads Gemini's response, uses an algorithm to convert the response to an audio file, and plays it to the user. Additionally, it also returns the regular text response from Gemini to ensure that nothing breaks in the process.



This implementation is optimal for simplicity on the developer end because it only affects one directory, `geminiv2/packages/cli`.

Plan for Testing Impact of Enhancement on Other Features

For testing the impact of our enhancement on other features, we will implement a very simple timer and computer resource monitor in the Text-to-Speech and Speech-to-Text packages during development. This will allow us to run rudimentary performance benchmarks on the affected areas of the software, such as how much RAM and CPU power is being used, but also the practicality of how long the user must wait while the packages convert the Gemini response to a playable audio file. Running tests with these metrics on the software will allow us to discover any unforeseen bugs and issues that our feature may cause to other components of the software. According to whether or not we determine the metrics of these tests to be acceptable, we will choose which open source Text-to-Speech and Voice-to-Text programs to use and implement them into the CLI repository.

Potential Risks

Security and Privacy Risks

Voice input is ambient, since a microphone captures everything in a room, not just the intended command. When a developer dictates a CLI instruction, the audio might also pick up a colleague reading out an API key, a background call discussing unreleased work, or sensitive source code. Under a cloud API implementation, all of that gets transmitted to the external cloud service. To manage this, the voice feature should be off by default and require

explicit opt-in. A clear disclosure should appear on first use, and all API communication should strictly enforce HTTPS.

Maintainability Risks

Audio processing libraries and device drivers for microphone and speaker access vary significantly across operating systems. For example, on Linux, behaviour depends on whether the system runs ALSA, PulseAudio, or PipeWire. Even using a cloud service implementation, the CLI still needs to capture the audio locally, which means relying on OS-level APIs. Every release will require audio-specific testing on all three platforms.

Performance Risks

Gemini CLI users frequently run resource-intensive background processes alongside the CLI itself. Continuous microphone polling or audio buffering adds overhead that can push an already-loaded machine into noticeably sluggish territory. The most effective fix would be a push-to-talk model: the microphone should only be active when explicitly triggered, getting rid of idle resource consumption.

Usability Risks (Transcription Errors)

Programming involves highly specific syntax, acronyms, file paths, and punctuation that standard STT engines frequently misinterpret. A high error rate would erode trust quickly, and a wrong transcription on a destructive command like 'git reset' can have catastrophic consequences. To prevent this, using a cloud API trained on technical commands would significantly improve accuracy. Additionally, the CLI should always display the transcribed text before executing any command, and destructive operations should require an explicit confirmation step, regardless of the input method.

AI Usage: Gustapo the Rat

We used Perplexity AI ("Gustapo the Rat"), again for Assignment 3 (A3), similar to what we did for A1 and A2. All interactions were restricted to a shared Space where searches were limited to course materials and a small whitelist of Gemini-CLI-related documentation. This ensured that all team members and Gustapo could view the same uploaded slides, assignment handouts, previous reports, Gemini CLI docs, and the A3 marking rubric. It also prevented the AI from incorporating random web content.

The A3 workflow was the same as the ones we had done before. First, we uploaded or refreshed all the important documents (course notes, A1/A2 reports, updated Gemini materials, and the detailed A3 rubric) so that Gustapo could "see" exactly what we were graded on. Next, we asked it to read our changing A3 draft and give us feedback on how clear the architecture was (with the new audio component), how consistent the conceptual and concrete diagrams were, and how well the "proposed improvement" and "lessons learned" sections covered their topics. We treated all of this feedback as hypotheses, just like we did before. Before we made any changes, we checked each suggestion against the actual repo, our diagrams, and the assignment text.

Main Applications in A3:

Diagram Review: Gustapo helped us fine-tune our box-and-arrow and sequence diagrams, ensuring that everything, from lifeline mappings to method calls, was consistent with the overall architecture. It also pointed out an error we had for the boundaries of the Gemini API.

Refining Use Cases: The tool was beneficial in clarifying our explanation of Use Case 1, which addresses the testing loop. However, we decided to ignore one suggestion regarding loop positioning because it didn't align with our architectural goals.

Sanity Check: In A3, we focused on adding the audio extension to Gemini CLI. To ensure we were on the right track, we utilized a whitelisted AI Space as a reviewer, which allowed us to validate our understanding of this new component and confirm that its responsibilities and dependencies aligned well with the existing architecture. Plus, it helped us avoid any external information that wasn't approved, keeping everything consistent with the course materials and Gemini CLI sources.

Marking Scheme Simplification: We simplified the marking rubric into distinct checkpoints and carefully checked our submission against them to ensure we met all requirements.

Quality Control: Gustapo essentially served as an "autograder," enabling us to compare its feedback with our primary sources. This process led to about a 15% improvement in our final text regarding explanations and comparisons, while we kept the diagrams and method names the same, based on our analysis of the repository and our use of the Understand tool. We always treated the AI-generated feedback as suggestions, rather than definitive answers.

This consistent methodology not only enhanced our understanding but also leveraged AI to increase efficiency, ensuring full traceability to the course materials.

Conclusion

Along with our proposed enhancement, there were several other ideas we chose not to explore in depth. One was a live visualization feature which could be implemented by streaming real-time data from the Gemini CLI agent to an interface that allowed users to monitor the steps it was taking. The purpose of which was to improve the overall user experience when it comes to processes that take a longer time. Another idea was to allow the CLI to create customized agents through a configuration system. Finally, we thought about transitioning Gemini into a hybrid CLI/GUI platform, however, this would involve a large restructuring of the core engine to allow both CLI and GUI frontends to interact with the same Gemini API. Because of this we found our first idea of audio integration to be our best choice for a proposed enhancement.

In conclusion, this report holds information on our proposal of a new audio implementation enhancement to the Gemini CLI AI agent in which both a speech-to-text and text-to-speech feature will be available. The first possible use case was where a user conversed vocally with the AI to test and debug a codebase. Using a sequence diagram we showed how an iterative feedback loop between the user, engine, and Gemini model performed testing, failure analysis, and automated fixing code until it passes all tests. The

second possible use case covered creating a file containing a program with requirements from a URL. The vocal command is sent to the audio interface, which then formats the request into a prompt for the Gemini API. For implementation alternatives, we explored implementing the audio feature as a subsystem of the core package, but found it created too much unnecessary back and forth that we do not see when allowing both the CLI and the core package direct access to an audio system. We also explored fully limiting an audio features integration, but found this implementation could cause latency issues due to the amount of communication that would be needed between the CLI and audio packages. The SAAM evaluation showed that a cloud-based architecture provided the best transcription accuracy, lower maintenance complexity, and better long-term scalability over a fully local solution. We used a box-and-arrow diagram to show how in low-level conceptual architecture the process added along the route to the core package. There is another box-and-arrow diagram to show the effect on the high-level conceptual architecture, specifically where the speech-to-text program belongs after it receives a response from the UI. We plan to test the impact of our enhancement by using a timer and a computer resource monitor to time both our text-to-speech and speech-to-text elements. We aim to also run performance benchmarks on important areas of software to ensure the implementation runs smoothly for the user. Though there are potential risks to this implementation there are also several solutions. For security and privacy, we will ensure the voice feature requires explicit and clear disclosures on use. For maintainability of device use throughout operating systems every release should require specific testing on all platforms. For performance risks due to continuous audio buffering would be a push-to-talk model. And finally, to ensure usability the CLI should display the transcribed text to confirm before executing any commands. Overall, we find that the speech-to-text and text-to-speech features would greatly improve accessibility and ease of access to Gemini CLI.

References

Staff Reporter. "Top 4 Ways Text to Speech Can Improve Accessibility and Productivity."

The Edinburgh Reporter, 4 July 2025,

theedinburghreporter.co.uk/2025/07/top-4-ways-text-to-speech-can-improve-accessibility-and-productivity/. Accessed 6 Apr. 2026.

ysg_adm22. "Assistive Technology." *Learning Disabilities Association of Ontario - LDAO*, 2025, www.ldao.ca/faq-items/assistive-technology/. Accessed 6 Apr. 2026.